

Prog Mob Project: Parkpal



Group: Taha Benslimane
Professor: Fabio Ciravegna
School Year: 2024-2025

Requirements

Functional Requirements

- The system **shall allow** the user to sign in using either email and password or Google authentication.
- The system **shall allow** the user to select one of their registered vehicles.
- The system **shall allow** the user to save their current parking location.
- The system **shall display** the saved parking location on an embedded map.
- The system **shall provide** navigation to the saved parking spot via Google Maps.
- The system **shall allow** the user to share the parking location through external apps such as Mail and WhatsApp.
- The system **shall allow** the user to view their past parking locations.
- The system **shall allow** the user to update their personal information.
- The system **shall allow** the user to add or delete vehicles from their profile.
- The system **shall allow** the user to change their login credentials.

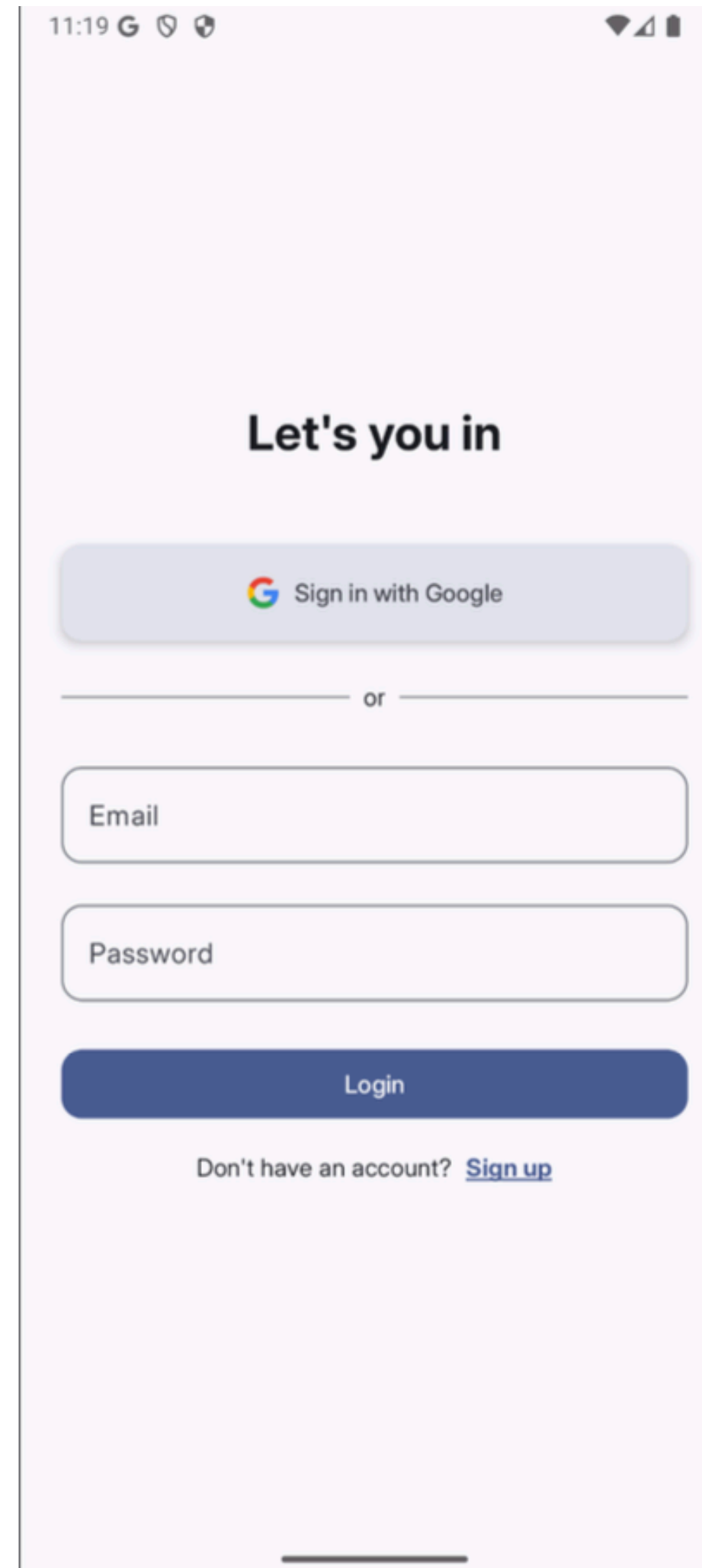
Non-Functional Requirements

- The app **shall handle** location permissions according to platform guidelines.
- The app **shall provide** an intuitive and easy-to-navigate user interface, including proper handling of the back button to return to the previous screen.
- The app **shall support** dark mode based on the system settings or user preferences.
- The app **shall persist** user data across app closures and device restarts.
- The app **shall follow** best practices for modularity and clean architecture principles.
- The app shall securely store and authenticate user credentials.
- The app **shall be implemented** following a clean, modular MVVM architecture.

Features

Authentication & Onboarding

- The user can be authenticated by Email/Password or by Google Sign-in
- On Sign Up, the user is requested to enter his personal information and the information of his vehicle



11:19

Let's you in

Sign in with Google

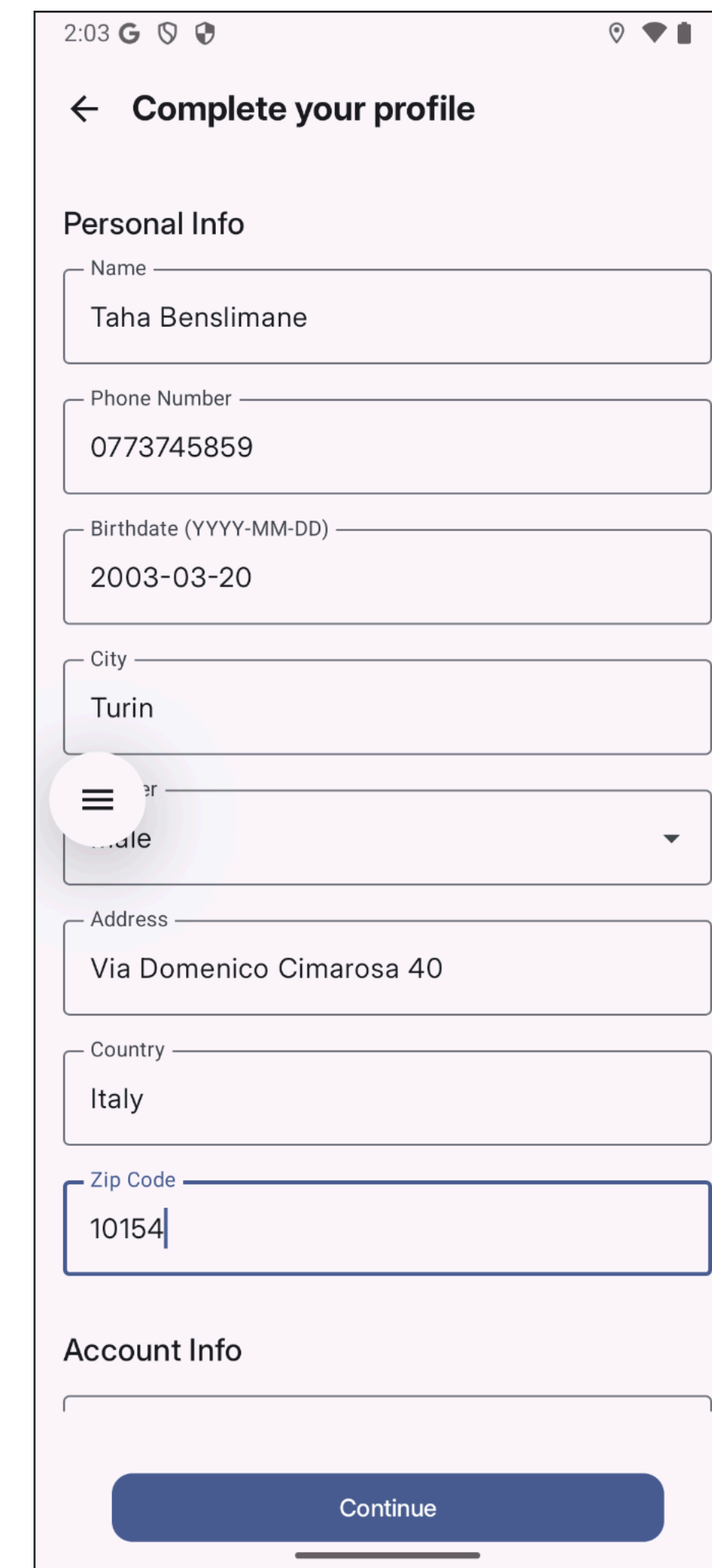
or

Email

Password

Login

Don't have an account? [Sign up](#)



2:03

← Complete your profile

Personal Info

Name
Taha Benslimane

Phone Number
0773745859

Birthdate (YYYY-MM-DD)
2003-03-20

City
Turin

Gender
Male

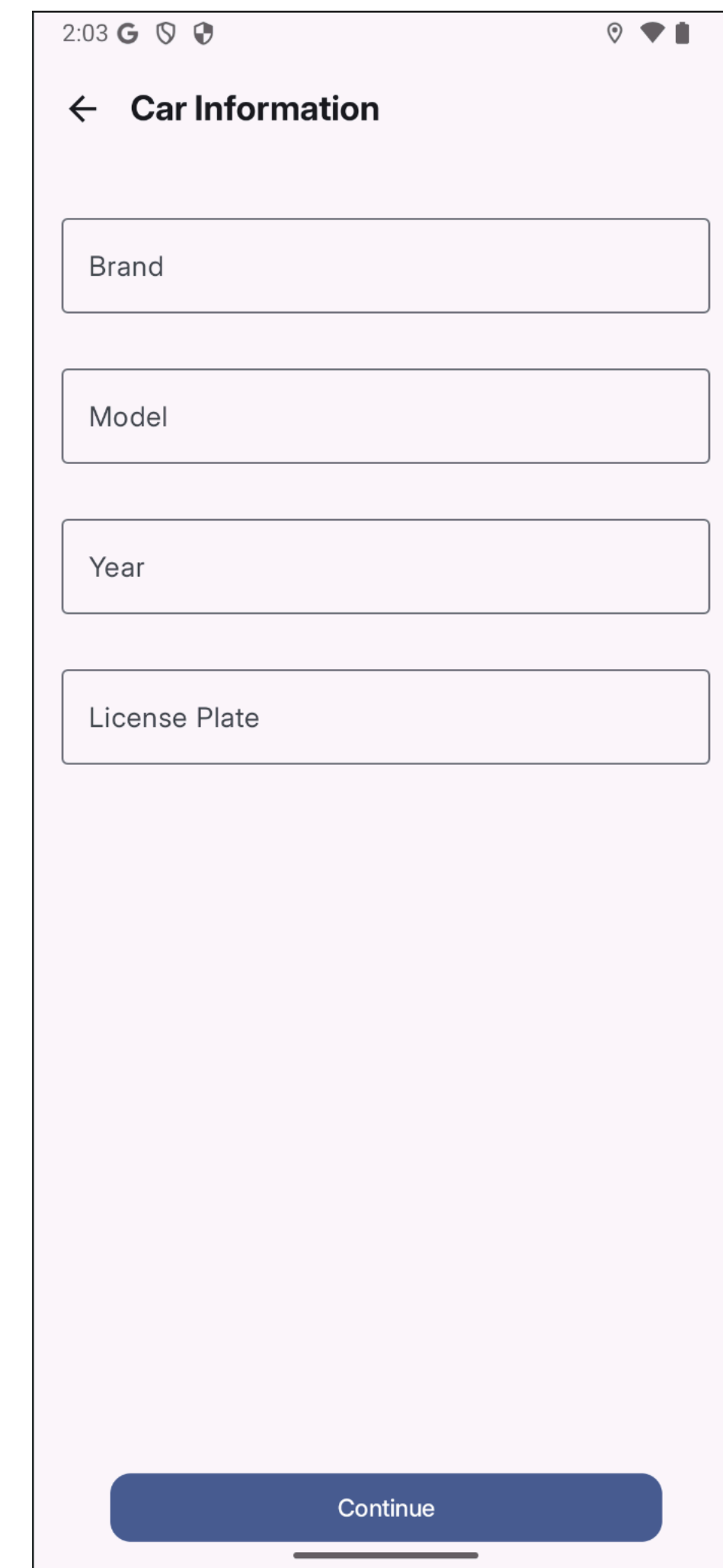
Address
Via Domenico Cimarosa 40

Country
Italy

Zip Code
10154

Account Info

Continue



2:03

← Car Information

Brand

Model

Year

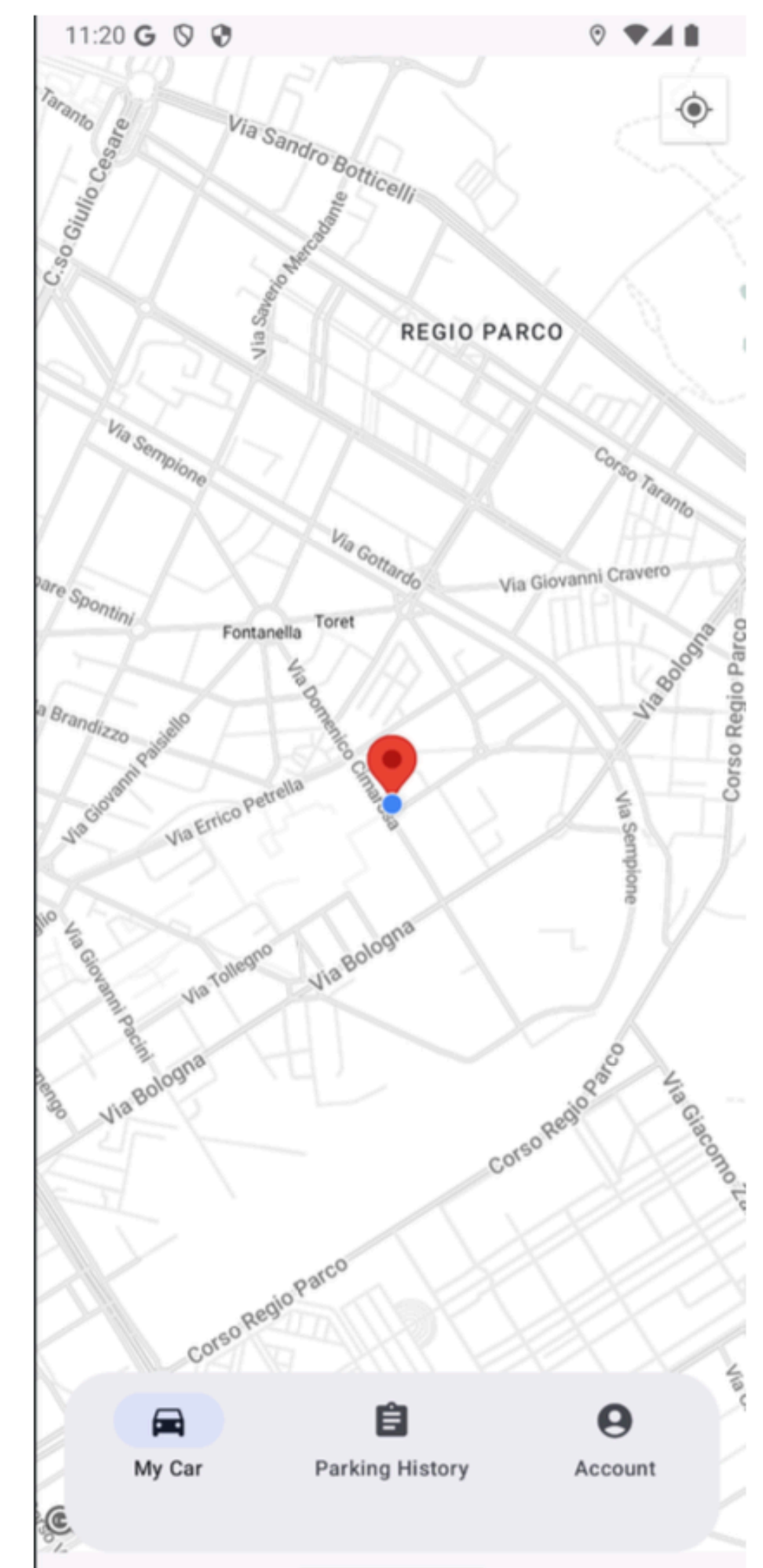
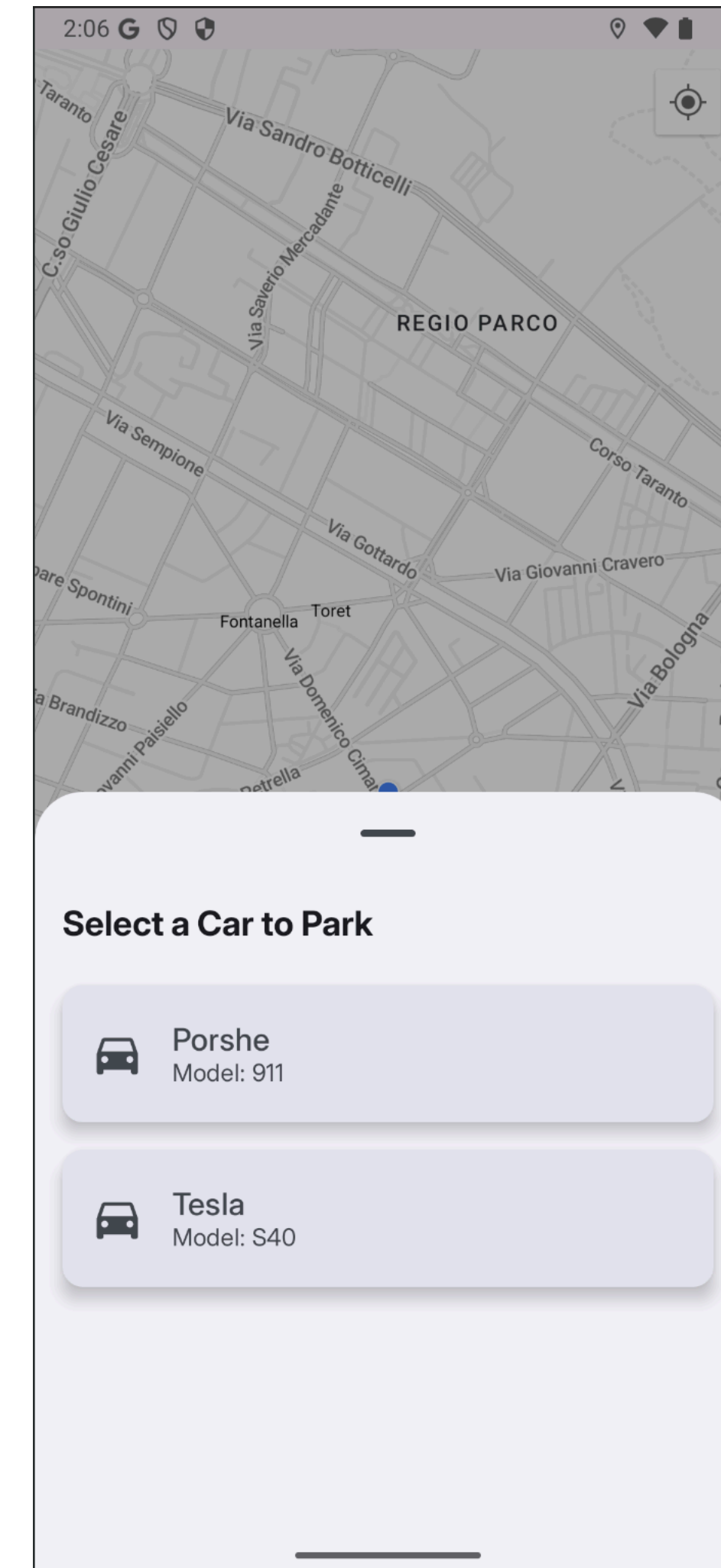
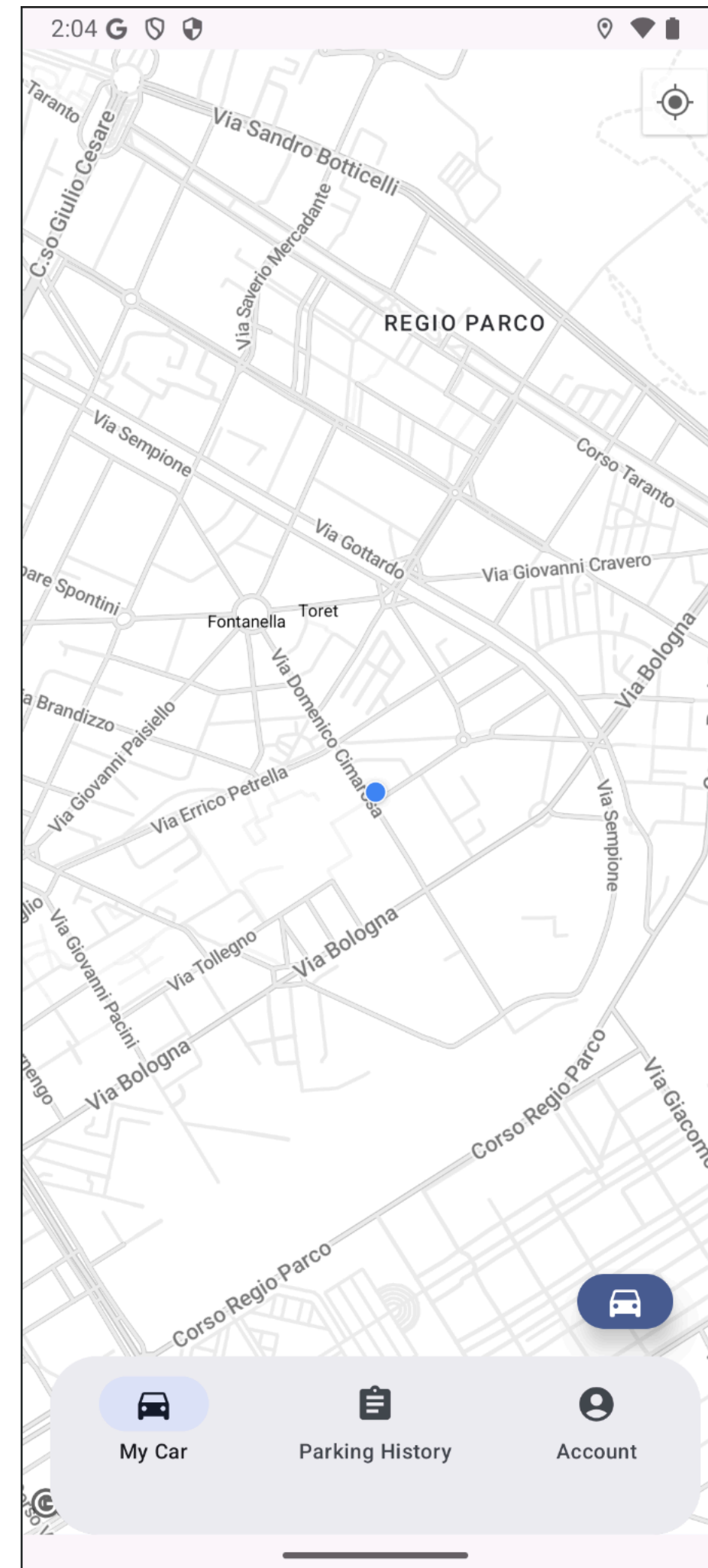
License Plate

Continue

Features

Home

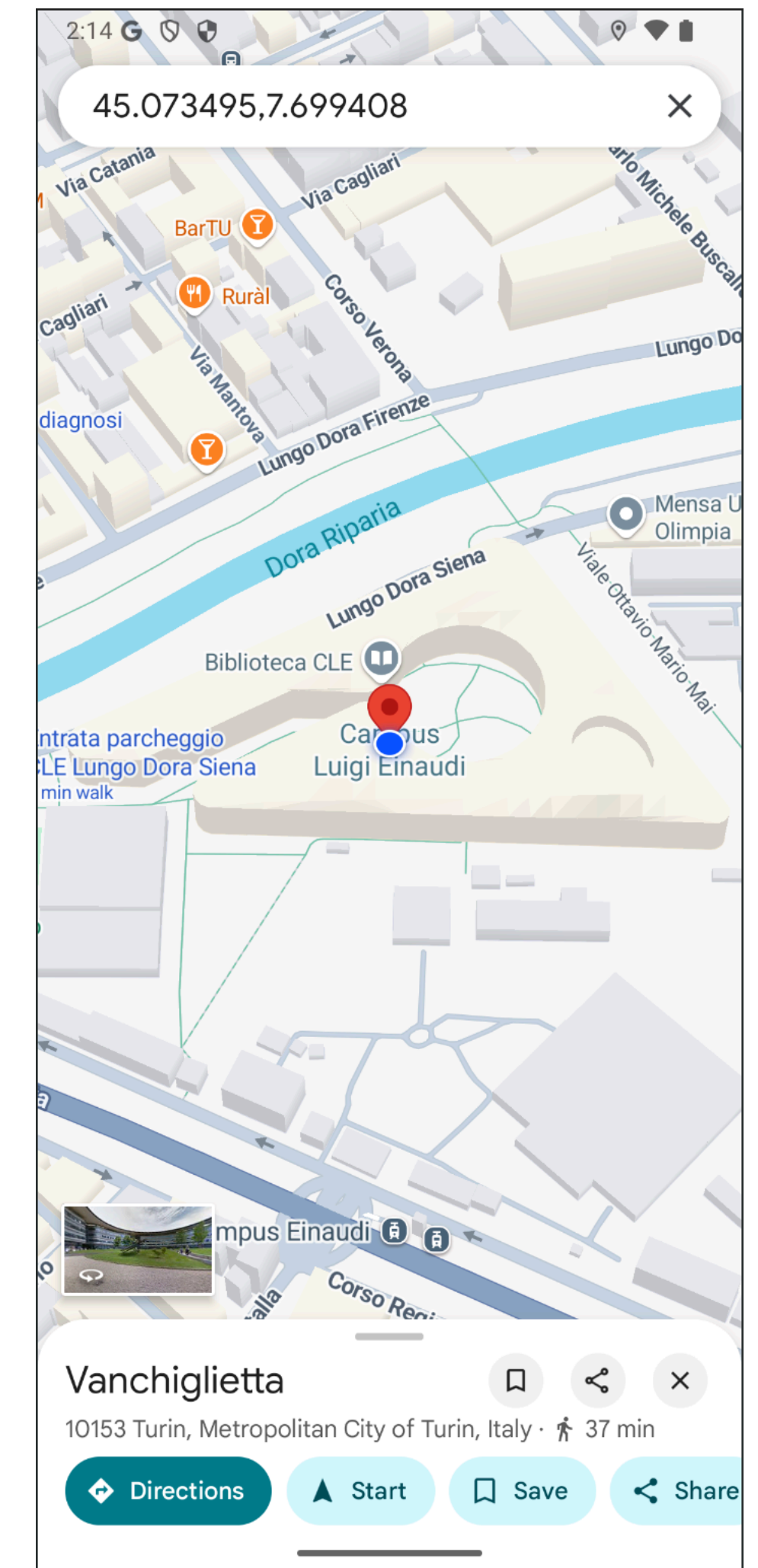
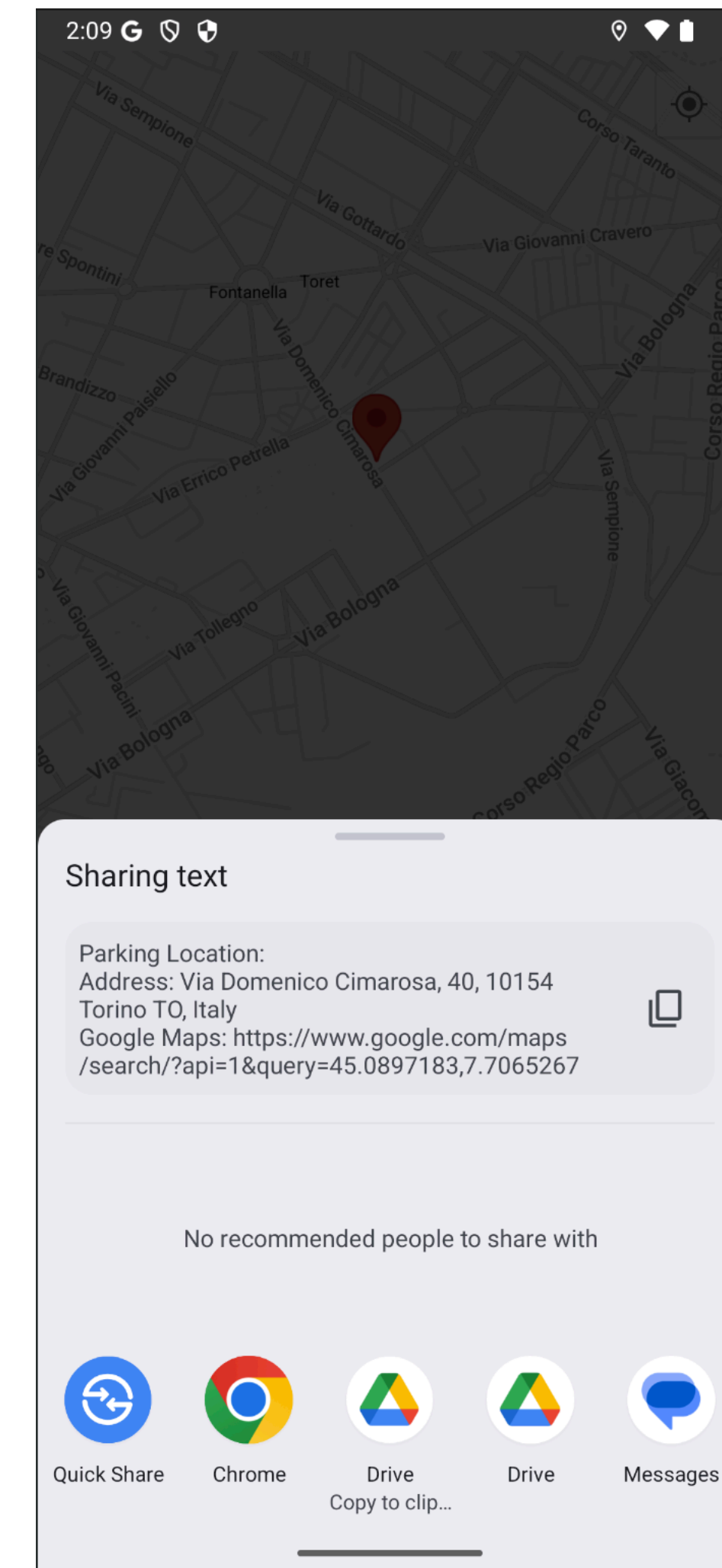
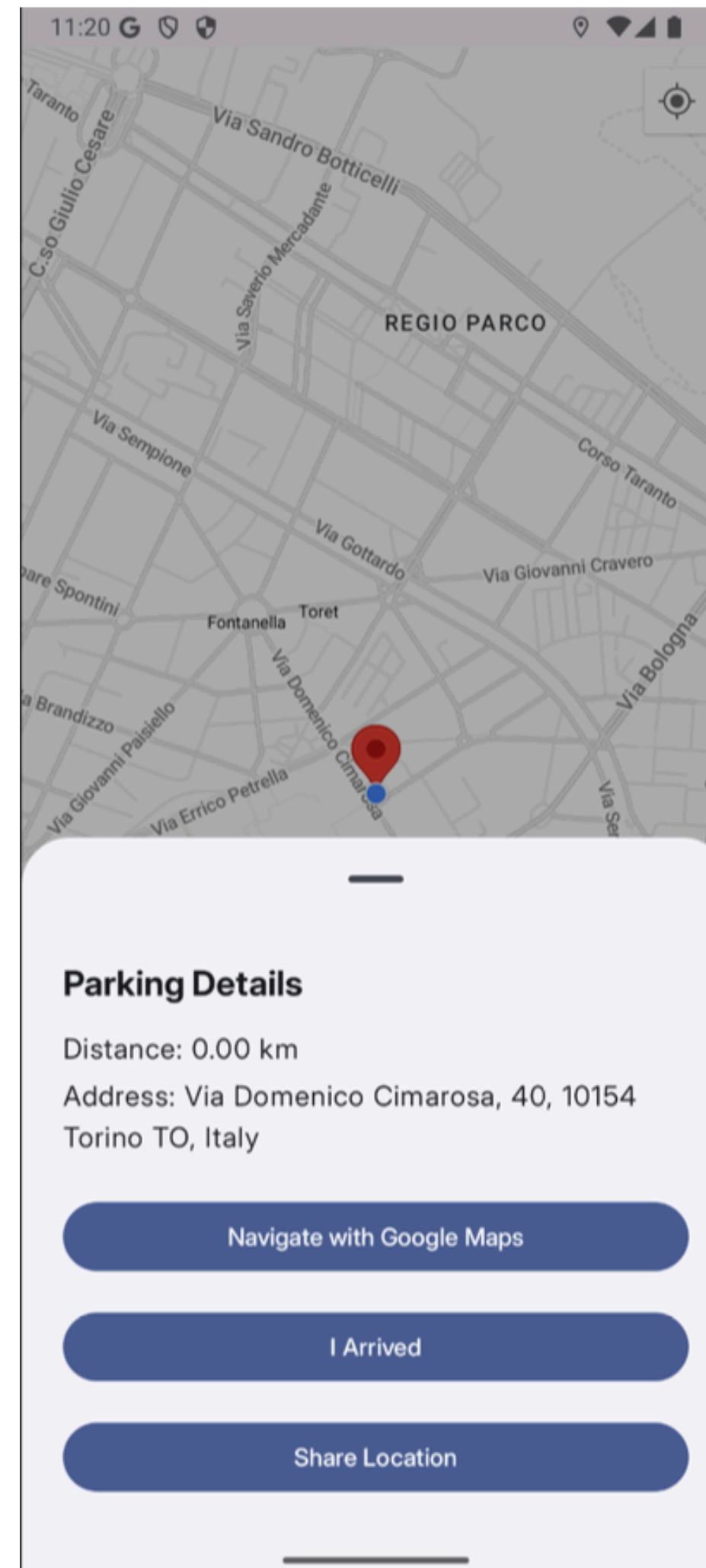
- The screen where the map is visualized
- The user clicks on the vehicle button to choose which vehicle to park, and then saves his parking location
- A marker is displayed on the map of his parking location



Features

Home

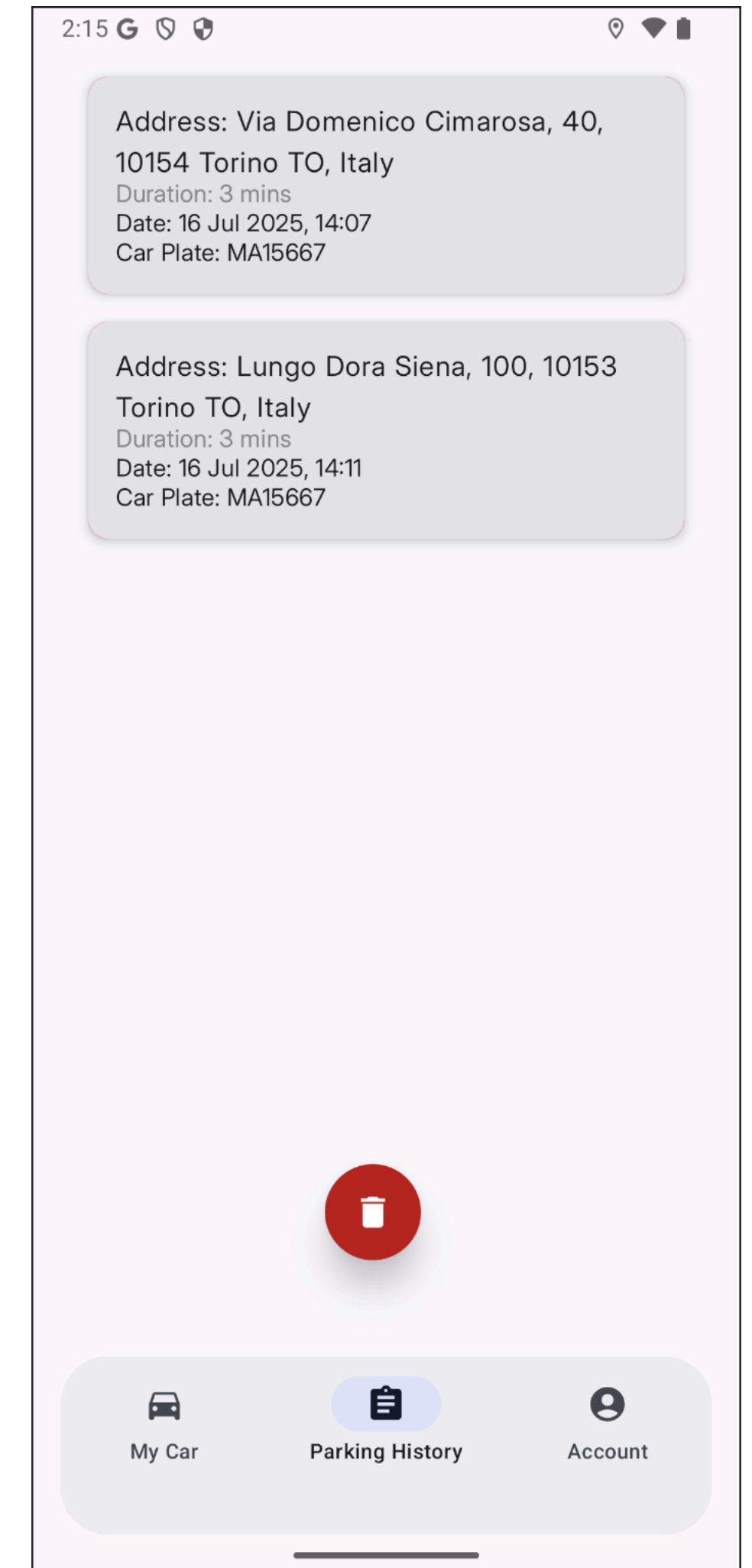
- On marker click, the user can
 - Visualize the address of the parking location and it's distance from his current location
 - Share the parking location with other people through Mail, Whatsapp, etc.
 - Launch navigation to the parking spot using Google Maps.
 - Indicate that he's arrived, the marker is then removed and the location is saved in the parking history



Features

Parking History

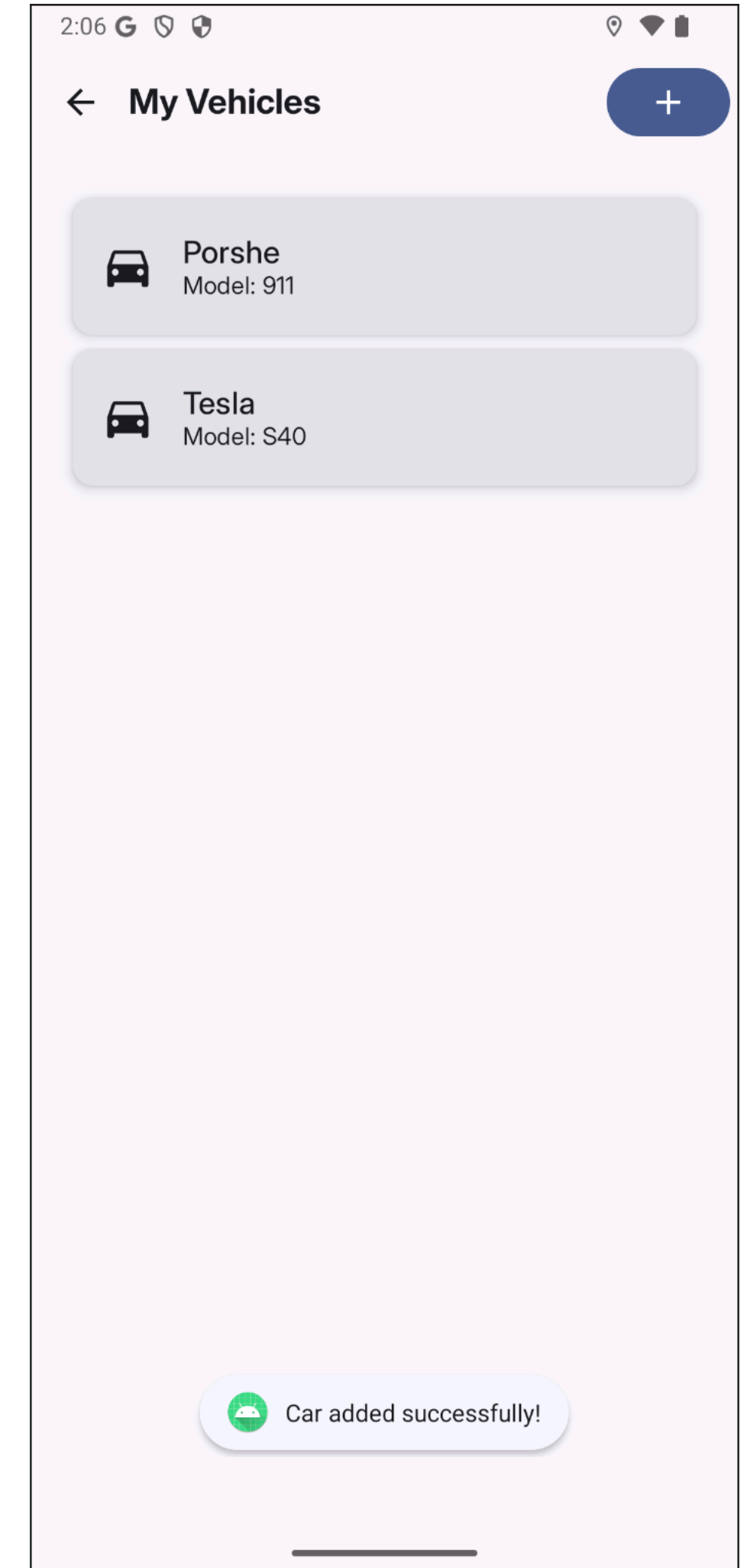
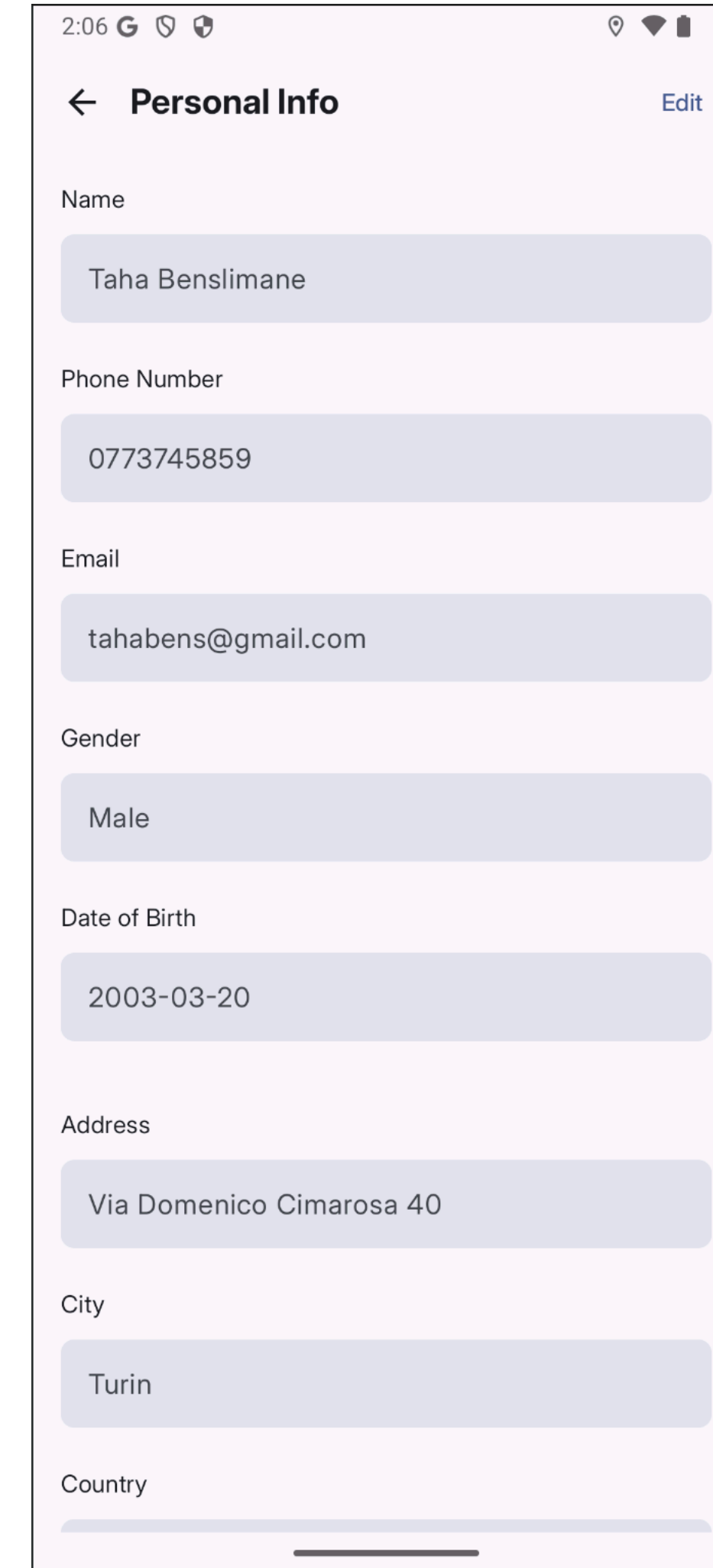
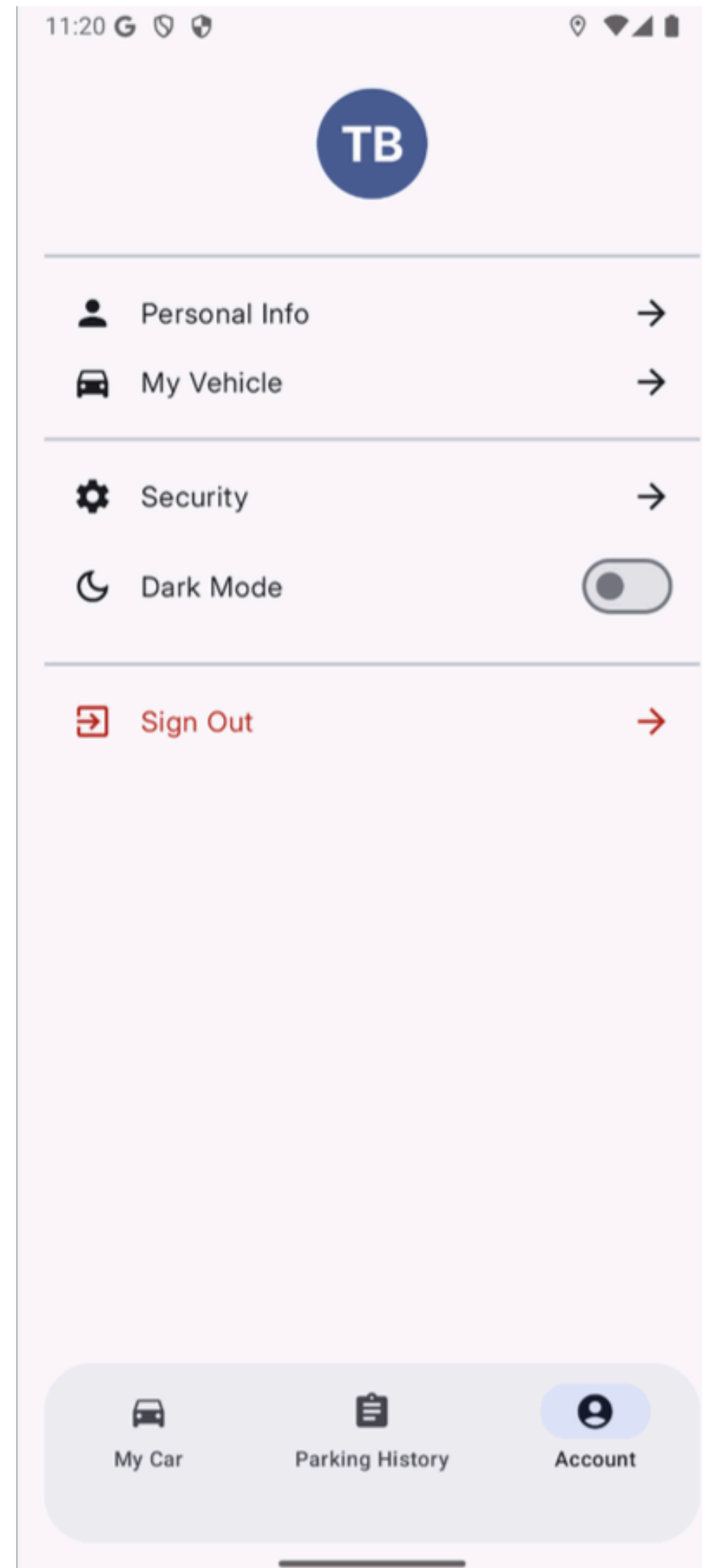
- The user can visualize his past parking locations
- The user can delete individual parking locations using a swipe-to-delete, and clear the whole parking history with the bin icon button



Features

Account

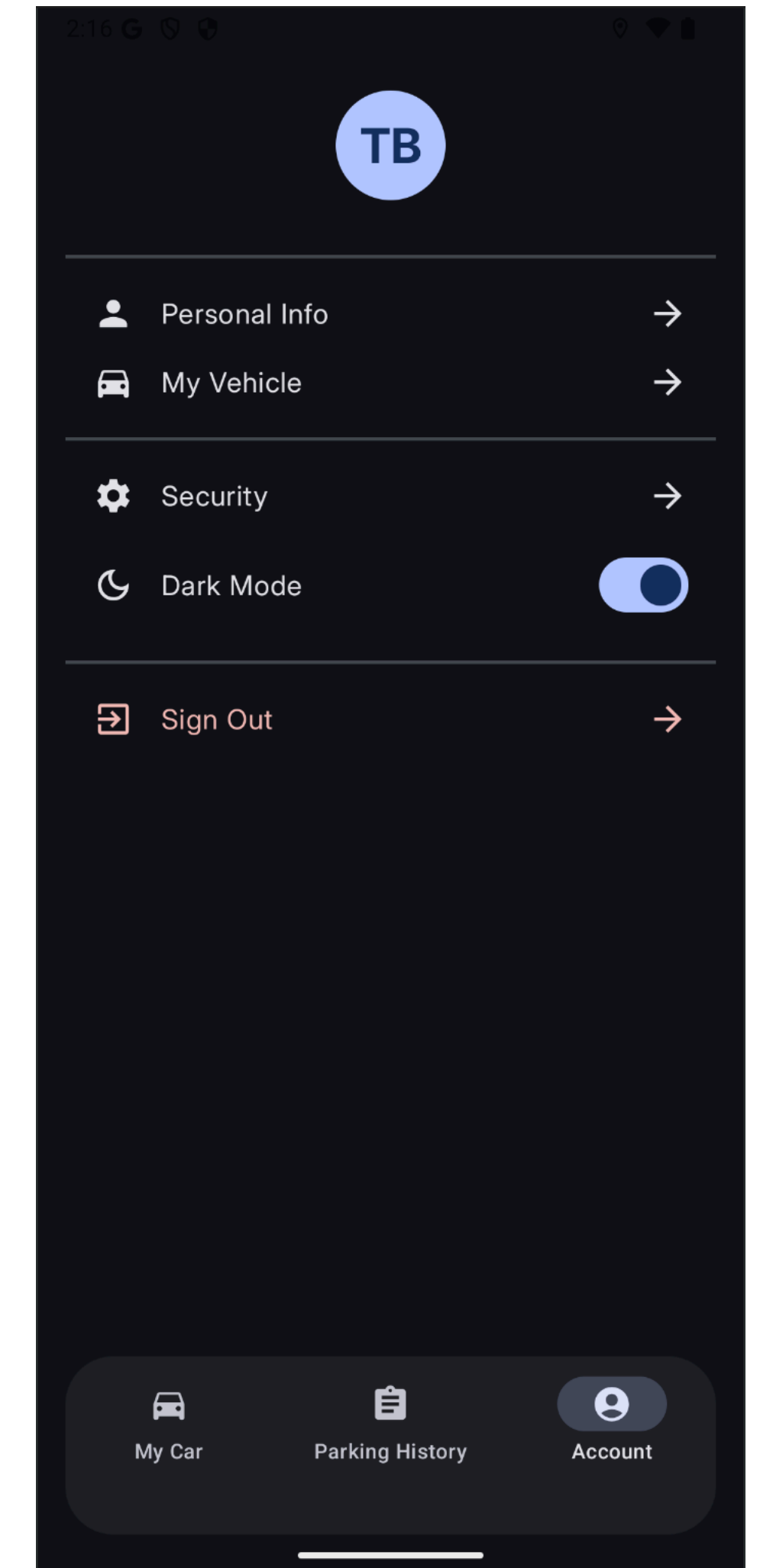
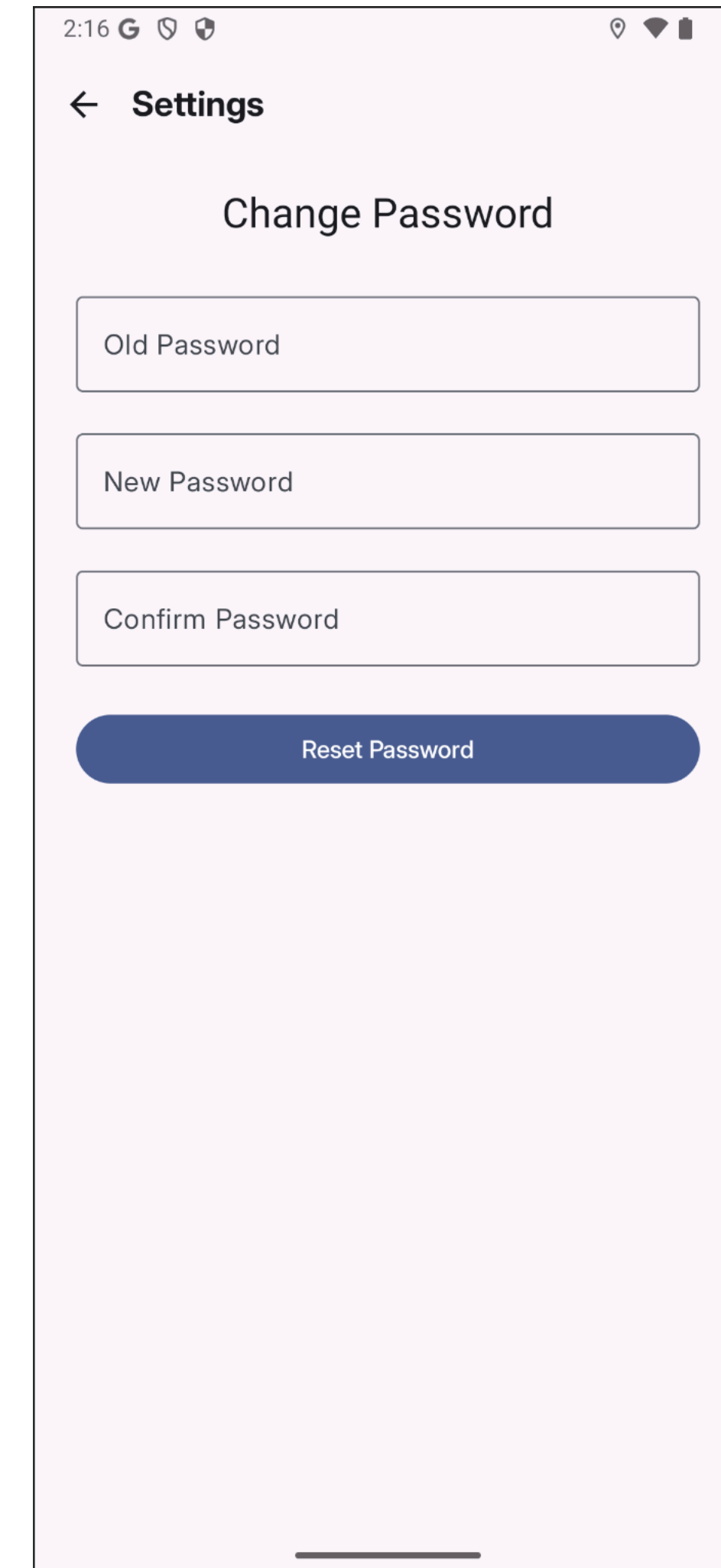
- In this screen the user can update his personal information
- The user can register new vehicles and delete existing ones using a swipe-to-delete



Features

Account

- The user can
 - Update his authentication credentials
 - Switch to dark mode
 - Sign out from the application



MVVM Architecture

View

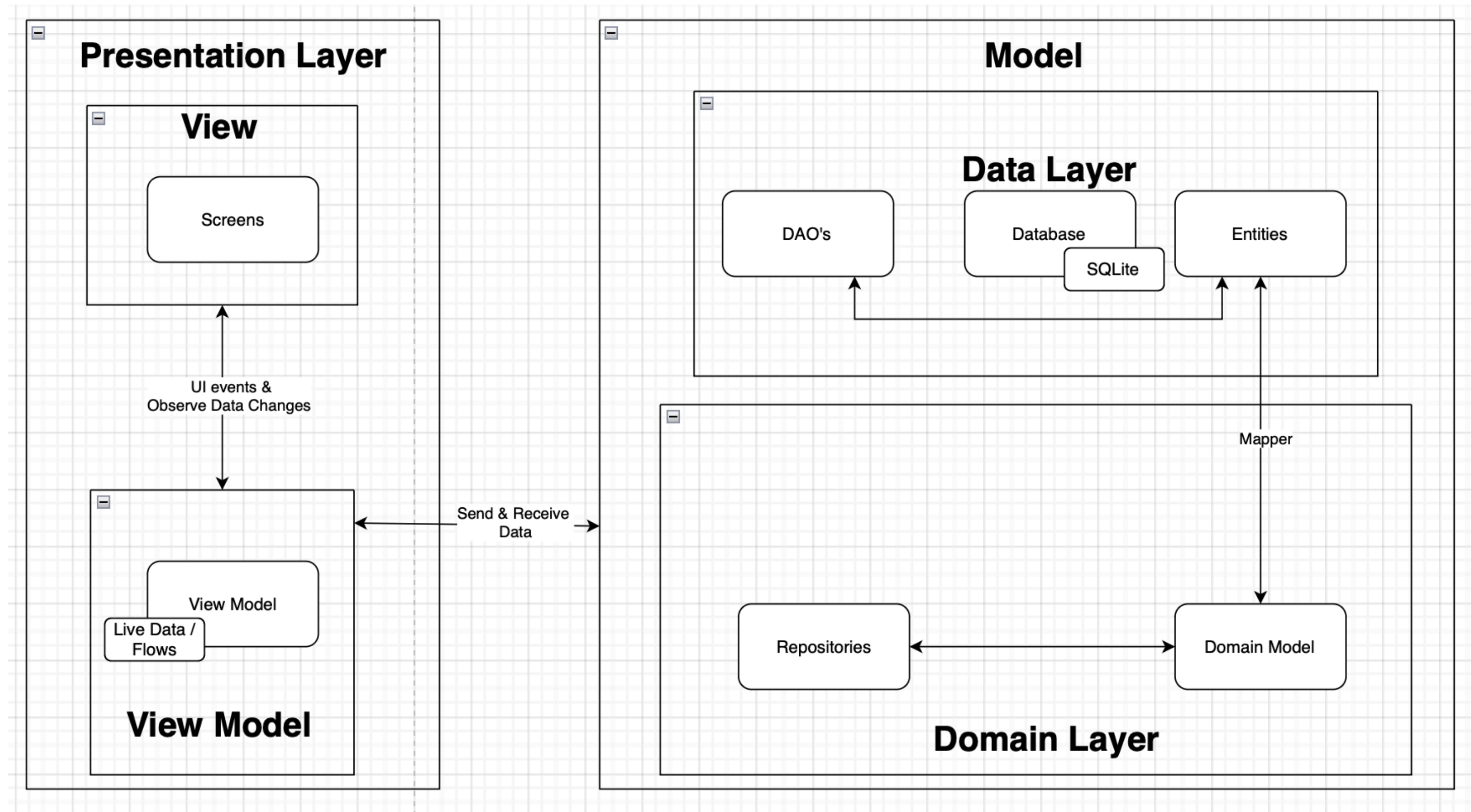
- Contains all the screens of the application

ViewModel

- Acts as intermediary between the View and the Model, handles the presentation logic and hoists the UI state

Model

- Handles the business logic and data access of the app



Clean Architecture

Presentation Layer

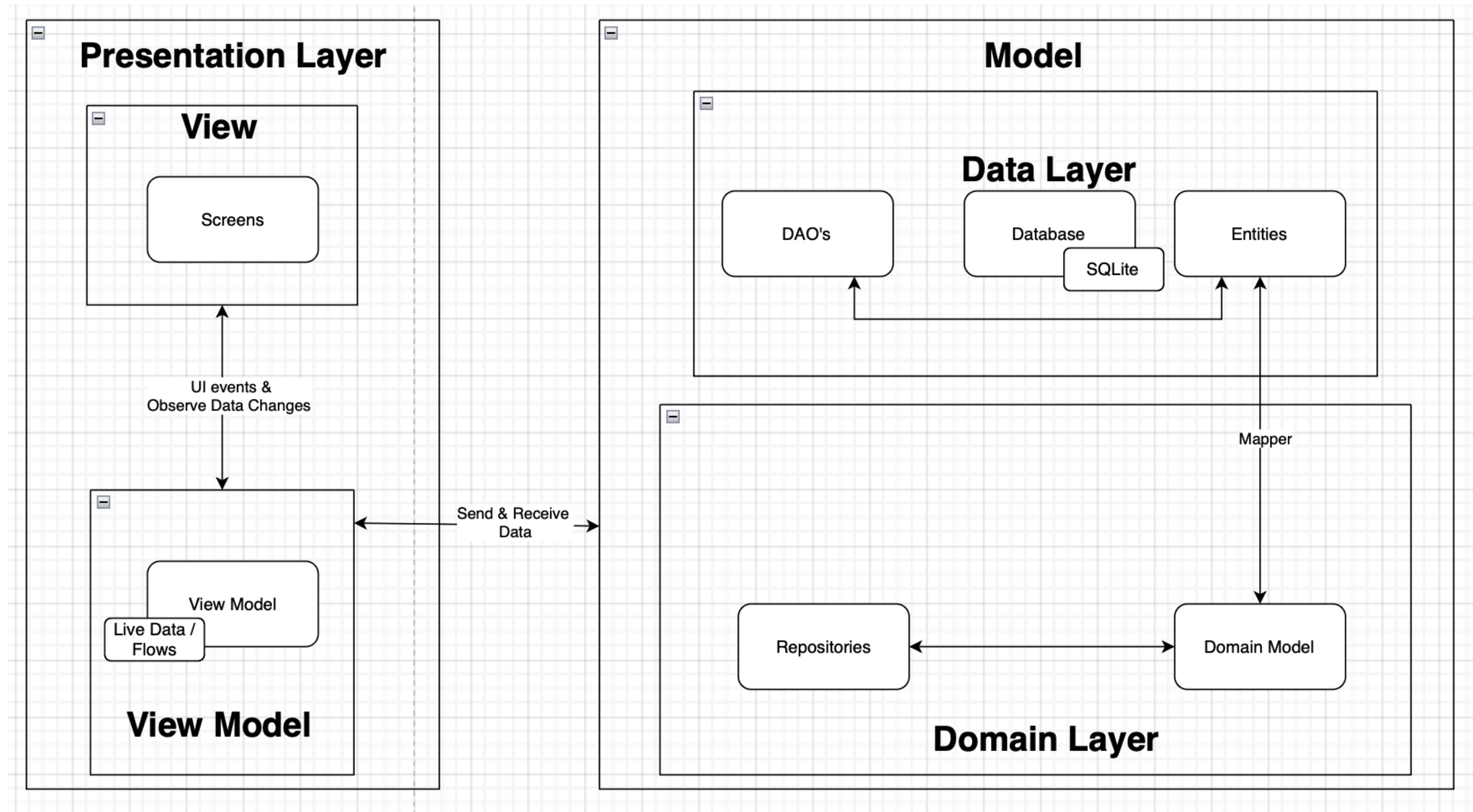
- Handles the UI and presentation logic
- Contains the ViewModels of the different screens
- Communicates with the Domain Layer

Domain Layer

- Main business logic
- Contains domain entities and the application's repositories
- Definition of the UserLocation Interface

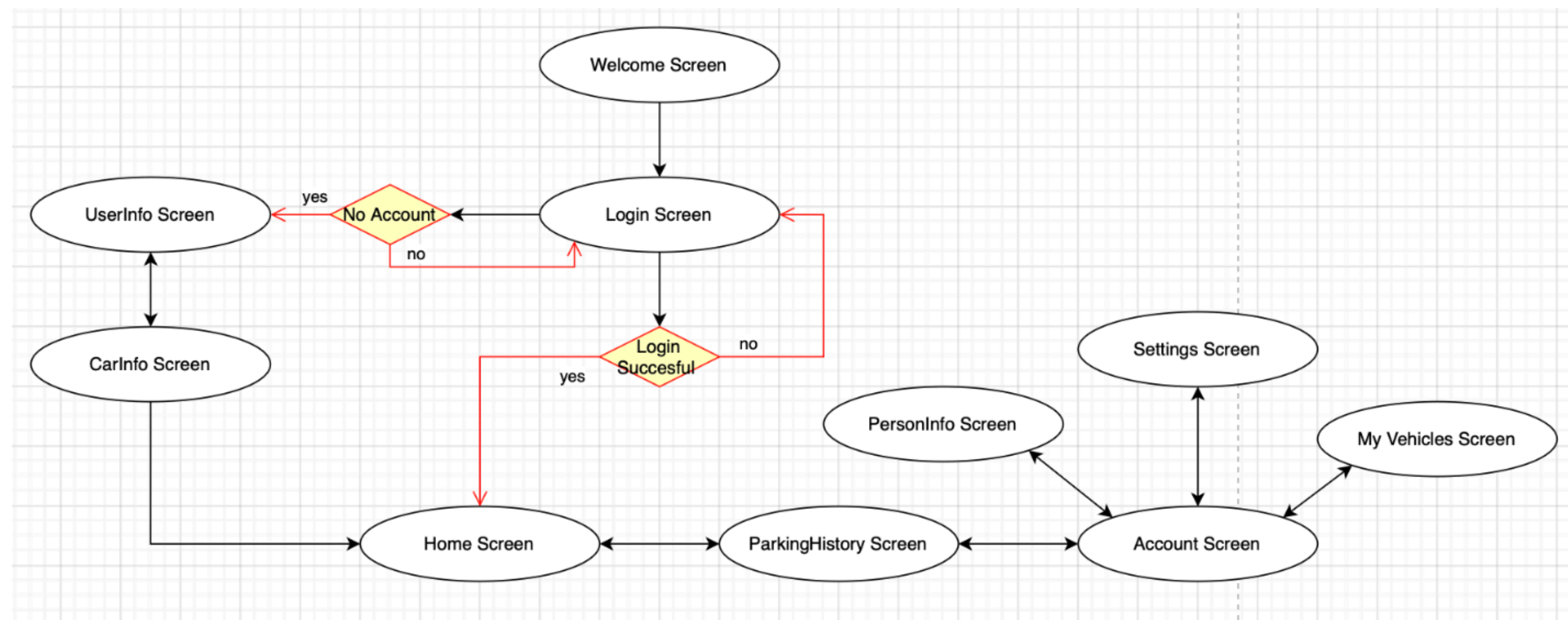
Data Layer

- Handles data access and interactions with the local database
- Implements the UserLocation Interface
- Contains Mappers to convert Room entities to domain entities and vice-versa



Navigation

- The app uses Jetpack Compose Navigation to manage transitions across onboarding, authentication, and main features. Navigation is structured in a single *AppNavHost* composable
- Navigation redirection is handled via callback functions passed to each composable screen. These callbacks invoke *NavController* to trigger the appropriate route transitions
- A single instance of each ViewModel is initialized once in then passed to the relevant screens to have a shared state
- The bottom navigation bar is conditionally shown based on the current route.
- Authentication state is observed to fetch or clear user/car data accordingly.



Compose

- Jetpack Compose is used as the app's modern UI toolkit, enabling fully declarative UI development.
- Screens are built as *@Composable* functions, allowing clean, modular, and testable UI code.
- Layouts are constructed using Compose UI elements like *Scaffold*, *Column*, *Row*, and *Modifier*.
- State management is handled via ViewModels, *collectAsState()* and *observeAsState()* for observing state flows and Live Data.
- Theme switching (dark/light) is integrated with a *isDarkTheme* flag passed into *AppNavHost*.

Dependency Injection

Dagger Hilt

- Dependency Injection Framework for Android
- It declares how dependencies are provided and managed across the app
- Makes unit testing easier by allowing dependencies to be swapped or mocked
- *@HiltAndroidApp* is applied to the custom Application class to initialize Hilt
- Dependencies are instantiated as Singletons and provided where needed
- *@HiltViewModel* annotation is used to inject dependencies into ViewModels

DI Package Structure

- DatabaseModule: Handles database instantiation and dependency injection
- RepositoryModule: Handles repositories dependency injection

ViewModels

- Manage and expose UI-related data to the Composables
- Act as state-holders, preserving UI state across configuration changes
- Use immutable *State*, *StateFlow* and *LiveData* to prevent external changes
- Use *viewModelScope* to launch coroutines for asynchronous tasks safely such as database access tasks and heavy computational tasks
- *MapViewModel* uses a sealed class *MapEvent* that contains map-related user actions and holds UI state in *MapState* data class

Permissions Handling

Necessary Permissions

- Access to user's location data (*ACCESS_FINE_LOCATION* and *ACCESS_COARSE_LOCATION*)

Permissions Handling Process

- The required permissions are declared in *Manifest.xml*
- In *HomeScreen*, location permissions are requested using *LauncherForActivityResult*
- When granted, the composable calls on the *ViewModel* method *startLocationUpdates()*

Room & Firebase

Room

- Android's persistence library
- Contains
 - Abstract *AppDatabase* class that extends RoomDatabase and contains a list of entities and methods to obtain the DAO's
 - DAO's that contain crud methods for basic CRUD operations and custom queries methods
 - *UserDAO, CarDAO, ParkingLocationDAO, ParkingHistoryDAO*
 - Entities that represent a database table defined as columns through the class attributes
 - *UserEntity, CarEntity, ParkingLocationEntity, ParkingHistoryEntity*

Firebase

- Cloud-based platform
- Provides secure authentication using Email/Password login and Google Sign-In
- Manages user session state and provides methods for sign-up, sign-in, sign-out, credentials update, etc. Implemented in *AuthViewModel*
- *GoogleSignInClient* wraps *FirebaseAuth* and *Credential Manager* for Google Sign-In, retrieves Google ID Token and sign-in with Firebase, returns a *GoogleUser*

Location Tracking

***UserLocationImpl* Object**

- Implements the *UserLocation* Interface
- Collects user location data using *FusedLocationProvider*
- Requests position every second
- Returns a Flow of *Location*

Fused Location Provider

- Part of Google Play Services Location API; provides efficient and accurate location tracking.
- Combines data from GPS, Wi-Fi, and cell towers to determine location with minimal battery usage

MAPS SDK

- Embeds interactive Google Maps into Android apps
- Supports map styling, markers, camera control and UI customization
- Offers real-time user location display and gesture handling
- Is implemented with Jetpack Compose using the GoogleMaps Composable
- The style of the map is imported from SnazzyMaps, and stored as JSON raw resource

Testing

Automated Tests

- Unit Tests validate presentation logic in ViewModels (user, car, parkingHistory)
- Test cases cover success, failures and edge cases
- Mockito used to mock dependencies
- Coroutines are used for asynchronous operations
- Instrumented Test for Sign-in

Manual Testing and Issues

- Through the manual testing performed through the development, a couple of minor issues have arisen in the app mainly with navigation to account screen and Email/Password sign-in that need to be fixed
- Also, there is a lot of room for enhancement and better handling of exceptions and relevant message display

Thank you for the attention!